Lab 2: Threat Modelling in Practice

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

In Lab 1 you enumerated the attack surface of the course VM: what listens, what is privileged, what is scheduled, what is exposed at rest. In this lab you turn that flat inventory into a **threat model**: a structured argument about *who* would attack the machine, *what* is worth protecting, *how* the attack proceeds, and *which weaknesses matter most*. You will name an attacker and value the assets, draw a data-flow diagram, apply STRIDE element by element, build an attack tree for one concrete goal, and end with a prioritised list. The deliverable is the model itself: a document you could hand to whoever operates the machine.

This lab is analysis, done on paper or in a diagram tool: you build a model, you do not exploit anything. **One small exception:** confirming the attacker's `sudo` grant with `sudo -l -U svc-report` is a read-only query that needs your `student` privilege for that one command; it inspects policy, it does not run an attack. Bring your Lab 1 notes; you cannot do this lab without them.

Contents

- Learning Objectives
- Background: three instruments
- Before the lab (at home, ~20 min)
- Part A: Attacker and assets
- Part B: Data-flow diagram of the machine
- Part C: STRIDE, element by element
- Part D: An attack tree
- Part E: Prioritise, and measure
- Reflection
- References
- Deliverables: Lab report 1 (individual)

Learning Objectives

By completing this lab, students will be able to:

1. Characterise an attacker (position, goal, motivation) and rank a system's assets by value.
2. Represent a system as a data-flow diagram with explicit trust boundaries.
3. Apply the STRIDE taxonomy element by element to derive threats systematically rather than by inspiration.
4. Build an attack tree that decomposes an attacker goal into concrete steps, and read off the cheapest **attack set** (the minimal sub-tree: cheapest child at OR nodes, all children summed at AND nodes).
5. Turn an unordered set of weaknesses into a *prioritised* model, and justify the ordering.
6. Derive a one-line security **policy** statement from a threat model, and tell it apart from the mechanism that enforces it.

These map to FUC outcomes LO2 (defining protection policies from assets, actors, motivations and threats) and LO4 (recognising a system's security posture).

Background: three instruments

Data-flow diagram (DFD). A DFD shows four things: **external entities** (actors outside your control), **processes** (running code), **data stores** (data at rest), and **data flows** (arrows between them). Over the top you

draw **trust boundaries**: dashed lines crossed whenever data moves between things at different privilege or trust levels. Threats concentrate on the boundaries.

STRIDE. A checklist of six threat categories, applied to each element of the DFD: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege. Its value is **systematic coverage**, not a guarantee of completeness: you ask all six of every element instead of only imagining the attacks you already know, which reduces predictable omissions but does not prove the model complete (threats from *composition* can still be missed).

Attack tree. A goal at the root, decomposed into sub-goals and finally into concrete actions at the leaves, with AND/OR nodes. Annotate leaves with cost or difficulty; the attacker's likely route is the **cheapest attack *set*** -- at an OR node take the cheapest child, at an AND node you need *all* children so sum them. That set is a single path only when the tree has no AND nodes; otherwise it is a sub-tree.

Before the lab (at home, ~20 min)

This lab is analysis, not setup, but come prepared: - bring your **Lab 1 attack-surface inventory** (you cannot do this lab without it); - re-read Lecture 2 (data-flow diagrams, STRIDE, attack trees).

Preparation questions (submit with Lab report 1, this lab's report): - **P1.** Name the four elements of a data-flow diagram. - **P2.** In one sentence, what does STRIDE give you that an ad-hoc list of ”problems” does not?

Part A: Attacker and assets

Before modelling *how* the machine is attacked, state *who* attacks it and *what* is worth protecting. A threat model with an unnamed attacker and unvalued assets is untestable.

Exercise 1: Attacker profile. The owner of this VM, the `student` account, is a full administrator. That is expected and **out of scope**: we do not model the legitimate owner attacking their own machine. Model instead an **unprivileged, non-owner foothold**, concretely the `svc-report` service account (the identity behind the port-8081 service you found in Lab 1). **Model `svc-report` specifically, not ”any low-privilege user”:** its exact identity is the point, because *this* account owns the service that reads `report.conf`, and *this* account carries the one narrow `sudo` grant. A generic ”unprivileged user” would share the lack of `docker` membership but not those two things, and the model turns on them, so if you generalise, say which properties you are keeping (non-owner, not in `docker`) and which are specific to `svc-report` (its read of `report.conf`, its sudo grant).

”Not a general sudoer” is not ”no `sudo` rights at all”, and the difference is the whole model. A service account is rarely given *nothing*; it is given *something narrow* that someone judged safe. `svc-report` is exactly that case: the course image grants it one passwordless `sudo` command through a `sudoers.d` rule. **Check it, do not take it from here:** run `sudo -l -U svc-report` and record what your machine actually reports, because modelling what is actually there, not what the design intends, is the habit this lab builds. Carry it into the model **as a permitted command whose reach you have not yet measured**. **Do not assume it is harmless because it is one command, and do not go exploiting it here**, that is Lab 6 Part D, and the point of *this* lab is to model, not to break. The open question to write down, exactly as Lab 1 had you do with `backup-tool`, is: *what can one permitted command actually reach?* You will answer it in Week 6, and the answer will surprise you. The modelling lesson holds regardless: a narrow permitted command is a capability of unknown size until you look, and ”one allowed program” is the recurring shape of privilege escalation.

Write down this attacker's:

- **position:** unprivileged local code execution as a non-owner: not in `docker`, and with the one narrow passwordless `sudo` grant the image ships, which `sudo -l -U svc-report` shows you exactly, whose reach you have not yet measured (an open question for your model, resolved in Week 6), plus network reach to the machine;

- **goal(s):** escalate privilege to `root`; read data they are not authorised to read; and/or disrupt the service (a denial-of-service goal, which the availability asset in Ex2 and the D column of STRIDE both depend on, so name it if your motivation includes disruption);
- **motivation:** one or two sentences: why would someone target a machine like this? (The credentials it stores, its use as a pivot to other systems, disruption of the service, …)

Exercise 2: Assets. List the machine's assets worth protecting: for example the credentials in `report.conf`, `root` privilege itself, the integrity of the maintenance script `rotate.sh`, and the availability of the report service. Rank them by value **to the attacker you defined in Exercise 1**, with one sentence of justification each. This ranking feeds your prioritisation in Part E.

Part B: Data-flow diagram of the machine

 One honest caveat before you draw the remote client. Lab 1 measured the service *listening* on `0.0.0.0:8081`, and only fetched it over `127.0.0.1`; it did **not** prove the VM is reachable from another machine (that depends on interfaces, routing, firewall and the hypervisor's network mode). So the **remote client is an explicit *assumption* of your model, not an inherited observation**, mark it as an assumption, exactly the discipline Lab 1's `0.0.0.0`-versus-reachable distinction taught. Modelling a plausible remote attacker is legitimate; presenting reachability as a measured fact is not.

Exercise 3. Draw a DFD of the course VM as you found it in Lab 1. Include, at minimum:

- external entities: a **remote client** on the network, and the **local user** `student`;
- processes: the **port-8081 service** (`report.service`, running as `svc-report`), the **root cron job**, and the **shell** an attacker drives from their foothold;
- data stores: the **credentials file** (`/etc/report/report.conf`), the **maintenance script** (`/opt/maint/rotate.sh`), the **setuid binary** (`/usr/local/bin/backup-tool`), and the **sudo policy** (`/etc/sudoers.d/`) that grants `svc-report` its one narrow command;
- data flows connecting them. At minimum: the flow you observed in Lab 1 where `report.service` **reads** `report.conf` on each request; the **response flow** where `report.service` returns data to the remote client, which in Lab 1 carried `db_user` back over HTTP; the **cron flow** where the root cron job **executes** `rotate.sh`; and the **privileged flow** where `svc-report` invokes its one permitted command **through** the sudo policy. Keep each process and each flow as a *separate* element: `root cron job` is a process, `cron job → rotate.sh` is a flow, and conflating the two is the conservation error this exercise checks for.

Then draw the **trust boundaries**. There are at least three worth marking: the VM's network edge (remote client service), the privilege boundary between the unprivileged foothold and `root`, and the boundary between the service account and the rest of the system.

3a. Submit the diagram. In two or three sentences, identify which boundary you expect to be the *most* attacked, and why.

Part C: STRIDE, element by element

Exercise 4. Build a STRIDE table. The rows below are a **starter** covering the seeded focus items; **extend the table with the remaining DFD elements from Part B** (at least the remote client and the attacker's shell) so that every element of your diagram appears. For each cell either write a concrete threat for that (element, category) pair, or ”n/a” with a word on why none applies.

Element	S	T	R	I	D	E
report.service (:8081) (process)						
root cron job (process)						

Element	S	T	R	I	D	E
cron job rotate.sh (flow)						
report.service remote client, `db_user` (flow)						
backup-tool (setuid)						
report.conf (data store)						
rotate.sh (data store)						
sudo policy: `svc-report` grant (data store)						
(add: remote client)						
(add: attacker shell / foothold)						

You are not required to know the *exploit* for each threat: several mechanisms arrive in later blocks. You are required to *name the threat correctly by category*. For instance, a world-writable script executed by root is a **Tampering** threat that enables **Elevation of privilege**; a world-readable credentials file is **Information disclosure**; the service echoing `db_user` over HTTP is also **Information disclosure**.

4a. Pick the three cells you judge most serious. For each, name the STRIDE category, the element, and one sentence on the harm if realised.

4b. STRIDE is a **completeness aid, not a guarantee** (it reduces predictable omissions; it does not prove the model complete, and threats that emerge from *composition* can still be missed). Name one threat you found *only because you worked through the grid systematically*: one you would probably have missed had you simply listed ”problems” as in Lab 1.

Part D: An attack tree

Exercise 5. Choose one attacker goal, both starting from the unprivileged foothold of Part A (not from the `student` owner):

- **G1:** obtain `root` on the VM, starting from the unprivileged `svc-report` foothold (note before you start: the root account on this image is **locked**, `/etc/shadow` shows `root:*`, no usable hash, so a route that only *reads* `/etc/shadow` is Information Disclosure, **not** a route to root; a real G1 leaf has to *run code* as root or *write* a file root will act on); or
- **G2:** read the contents of `report.conf`, starting from the unprivileged foothold. **G2 has a subtlety that makes it the more interesting goal: `svc-report` can *already* read `report.conf`, because the service it runs reads it on every request, that access is inherited from the account's legitimate identity, not granted by the file's `0644` mode.** So for the `svc-report` attacker the mode is *not* the cause, and ”make it not world-readable” does *not* close G2 against them. Frame G2 as: what does a compromise of the service identity hand the attacker, and what *additional* separation (a secret only a narrower identity can read, a vault, a per-request credential) would be needed to deny it? The `0644` mode is the cause only for a *different, unrelated* local user, which is a second, weaker attacker worth naming.

Build an attack tree for your chosen goal. Root = the goal. Decompose into sub-goals with AND/OR nodes, down to leaves that are concrete actions against *this* machine (referencing the specific items from your DFD). Annotate each leaf with a rough difficulty (low / medium / high) and whether it needs a mechanism you have not yet been taught (mark those ”deferred”).

5a. Read off the cheapest way to satisfy the root goal, and **be careful how you compute ”cheapest”, because it is not simply ”the shortest path”.** In an **OR** node you take the *cheapest child*; in an **AND** node you need *all* the children, so you *sum* their costs. The result is a minimal **sub-tree** (an attack set), which is a single path only when the tree has no AND nodes. State your answer as the ordered steps of that minimal set. (For example, the cron route is an **AND** of ”write to `rotate.sh`” and ”wait for root to run it”, two leaves and a delay, whereas the `svc-report` sudo grant, *if its one permitted command turns out to reach a shell*,

would be a single OR-branch with no waiting. Whether it does is the open question from Part A; your cost estimate should carry that uncertainty as a ”deferred” leaf rather than assume the answer.)

5b. Your tree probably contains at least one ”deferred” leaf (a step whose mechanism is taught later: setuid abuse, editing the cron-run script). Does the *shape* of the cheapest path depend on that deferred step, or does a path exist entirely from things you already understand? (For G2, note that `report.conf` is world-readable, so any foothold already reads it: what does that tell you about how ”hard” G2 really is?) What does the answer reveal about how much of this machine's risk is already legible to you after two labs?

Part E: Prioritise, and measure

Exercise 6. Produce a **ranked** list of the machine's weaknesses, most to least urgent to fix, drawing on Parts C and D. For each item give: a one-line description, the STRIDE category, the single change that would remediate it (name it; you need not know how to apply it yet), and a one-sentence justification of its rank.

6a. Defend your top-ranked item against the second: why does the first deserve attention before the second? A defensible ordering, argued, is worth more than a ”correct” one asserted.

6b. Lab 1 claimed attack surface can be treated as a *measurable quantity*. Propose one simple, countable metric for this machine's attack surface (for example: number of network-reachable services running as root; number of world-writable files executed by a privileged context). Give its current value for the VM, and state what it would become if your top-ranked remediation were applied.

6c. You have just named a mechanism for your top-ranked item. Lecture 2 asked you to write the **policy line first**, or the mechanism quietly becomes the policy. Recover it now: write, in **one line**, the statement your remediation assumes about **what is allowed on this machine, and by whom**, in the form of the lecture's example (*no unprivileged user may modify anything that root executes*). Then name **one other mechanism** that would enforce the same line (a different permission, a different owner, removing the program altogether) and say in one sentence what each of the two allows that the other does not. If the two differ, the line was doing work the mechanism alone was not.

A policy line says who may do what, not which flag to set. ”`rotate.sh` should not be world-writable” is your mechanism restated; ”no unprivileged user may modify anything root executes” is the policy it serves, and it also forbids things you have not thought of yet.

Reflection

Your reflection goes into Lab report 1, as an appendix, beside the reflection from Lab 1, the other lab in this block. It is read under C3; see `rubrics.md`.

In a short piece (200 to 300 words):

1. A threat model built *after* an attack tends to justify what the attacker already did. This lab asked you to build the model *before* exploiting anything. What does modelling-first buy a defender that modelling-after does not?
2. STRIDE, DFDs and attack trees are three different lenses on the same machine. Where did they *disagree* or surface different things, and does that make the set redundant, or complementary?

References

- A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014 (STRIDE and DFDs).
- Microsoft, ”The STRIDE Threat Model”, https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
- B. Schneier, ”Attack Trees”, *Dr. Dobb's Journal*, 1999.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: Ch. 1.

Deliverables: Lab report 1 (individual)

Lab report 1 is the threat-model document, submitted **individually** even though the lab work is done in your group of three. It comprises: the attacker profile and ranked assets (Part A), the DFD with trust boundaries (Part B), the STRIDE table extended to every DFD element plus your three most-serious cells (Part C), the attack tree and its cheapest attack set (Part D), and the ranked remediation list with your defence of the top item, the surface metric, and the policy line your top remediation assumes with its second mechanism (Part E). Include your answers to this lab's preparation questions P1 and P2, and, as an appendix, your Lab 1 attack-surface inventory with Lab 1's preparation questions: the P1 screen capture, as an image and not as pasted text, which is your proof that the machine was running before the session, and your P2 answer. **Lab report 1 also carries the reflections from both labs in this block**, Lab 1's and this one's, as a further appendix: they are read under C3, as `rubrics.md` sets out.

Marked on a compact rubric (soundness of the attacker/asset framing; correctness of the DFD and boundaries; systematic use of STRIDE; validity of the attack tree; quality of the prioritisation argument; and whether the policy line is a policy rather than its mechanism restated). Due on Blackboard by **Sunday 11 October**, one week after Block 1 ends; the exact time is the one on the Blackboard assignment, and the late-report rule in the rubrics applies.

(No screencast for this lab. Unlike the hands-on labs, Lab 2 is a written analysis with no run to record; report 1 is assessed on the model itself. The Lab 1 screencast, which narrates the enumeration this model consumes, is the visual support for the pair.)